Introduction to Theoretical Computer Science

CS2490 - Session 25: Pushdown Automata

Pierre Cagne

Fall 2026

October 16, 2026

Last time

We introduced context-free grammars (CFG): every production rule has a single variable on the left-hand side.

Last time

We introduced context-free grammars (CFG): every production rule has a single variable on the left-hand side.

We have seen a CFG that generates all the palindromes of even length.

Last time

We introduced context-free grammars (CFG): every production rule has a single variable on the left-hand side.

We have seen a CFG that generates all the palindromes of even length.

We have seen a CFG that generates all the binary words with same number of 0s and 1s.

Last time

We introduced context-free grammars (CFG): every production rule has a single variable on the left-hand side.

We have seen a CFG that generates all the palindromes of even length.

We have seen a CFG that generates all the binary words with same number of 0s and 1s.

We ended up on a CFG that generates all the arithmetic expressions with addition and multiplication: $(\{0, 1, \ldots, 9, (,), +, \times\}, \{E, I\}, E, P)$ with P consisting of

$$E \to E + E \mid E \times E \mid (E) \mid I, \quad I \to 0 \mid 1 \mid \ldots \mid 9 \mid 0I \mid 1I \mid \ldots \mid 9I$$

Example of an ambiguous grammar (cont'd)

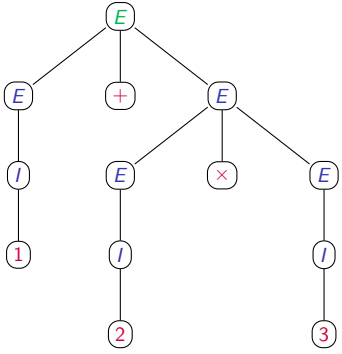
The previous grammar is ambiguous.

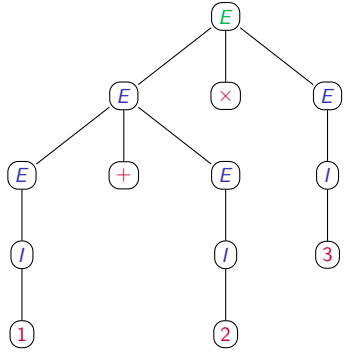
Example of an ambiguous grammar (cont'd)

The previous grammar is ambiguous. For example, the word $1 + 2 \times 3$ has two different derivation trees:

Example of an ambiguous grammar (cont'd)

The previous grammar is ambiguous. For example, the word $1 + 2 \times 3$ has two different derivation trees:





Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$
- $E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$
- $E \Rightarrow E \times E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$
- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$
- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow I + E \times E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$
- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow I + E \times E \Rightarrow 1 + E \times E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$
- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow I + E \times E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$
- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow I + E \times E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$
- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow I + E \times E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow$

Exercise

Find the two leftmost derivations of $1 + 2 \times 3$ corresponding to the two derivation trees of the previous slide.

Answer:

- $E \Rightarrow E + E \Rightarrow I + E \Rightarrow 1 + E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$
- $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow I + E \times E \Rightarrow 1 + E \times E \Rightarrow 1 + I \times E \Rightarrow 1 + 2 \times E \Rightarrow 1 + 2 \times I \Rightarrow 1 + 2 \times 3$

Summary

Summary

- Context-free grammars are a restriction of general grammars where production rules have a single variable on the left-hand side.

Summary

- Context-free grammars are a restriction of general grammars where production rules have a single variable on the left-hand side.
- Context-free grammars generate context-free languages, which are more general than regular languages.

Summary

- Context-free grammars are a restriction of general grammars where production rules have a single variable on the left-hand side.
- Context-free grammars generate context-free languages, which are more general than regular languages.
- Derivation trees provide a graphical representation of derivations in context-free grammars.

Summary

- Context-free grammars are a restriction of general grammars where production rules have a single variable on the left-hand side.
- Context-free grammars generate context-free languages, which are more general than regular languages.
- Derivation trees provide a graphical representation of derivations in context-free grammars.
- Leftmost and rightmost derivations are special types of derivations that are in one-to-one correspondence with derivation trees.

Summary

- Context-free grammars are a restriction of general grammars where production rules have a single variable on the left-hand side.
- Context-free grammars generate context-free languages, which are more general than regular languages.
- Derivation trees provide a graphical representation of derivations in context-free grammars.
- Leftmost and rightmost derivations are special types of derivations that are in one-to-one correspondence with derivation trees.
- Ambiguity in context-free grammars occurs when a string can be generated by multiple distinct derivation trees.

Finite state machines for context-free languages?

Are there finite state machines that are to context-free languages what DFA/NFA are to regular languages?

Finite state machines for context-free languages?

Are there finite state machines that are to context-free languages what DFA/NFA are to regular languages?

Answer: Yes, they are called pushdown automata (PDA).

Finite state machines for context-free languages?

Are there finite state machines that are to context-free languages what DFA/NFA are to regular languages?

Answer: Yes, they are called pushdown automata (PDA). Morally speaking, they are automata with some kind of memory.

PDA: intuition

Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.

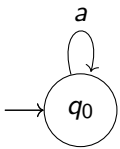
PDA: intuition

Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.

$\longrightarrow$ Memory: 0

PDA: intuition

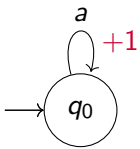
Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.



Memory: 0

PDA: intuition

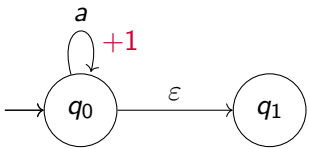
Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.



Memory: 0

PDA: intuition

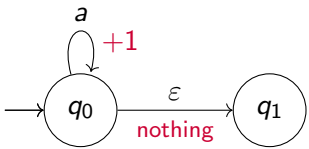
Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.



Memory: 0

PDA: intuition

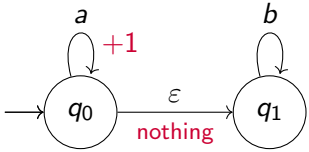
Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.



Memory: 0

PDA: intuition

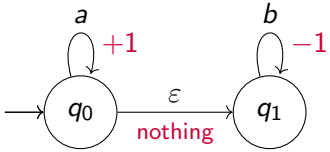
Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.



Memory: 0

PDA: intuition

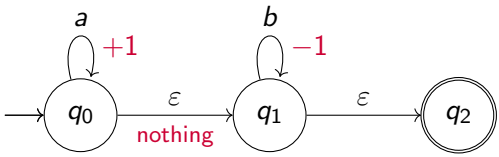
Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.



Memory: 0

PDA: intuition

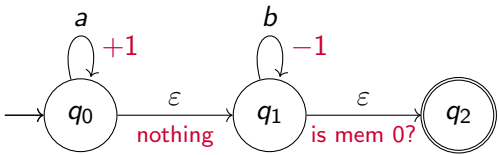
Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.



Memory: 0

PDA: intuition

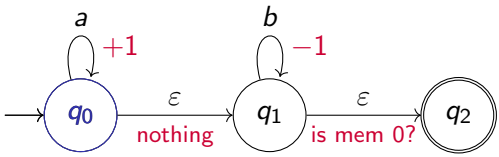
Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.



Memory: 0

PDA: intuition

Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.

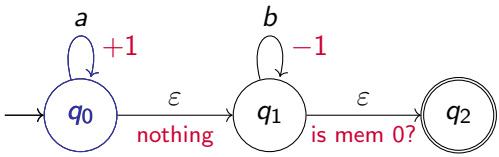


Memory: 0

Run it on input *aabb*.

PDA: intuition

Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.

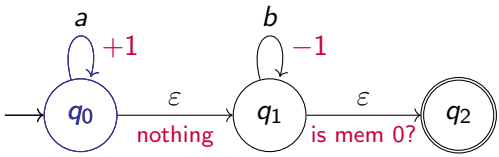


Memory: 1

Run it on input $\mathbf{a}abb$.

PDA: intuition

Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.

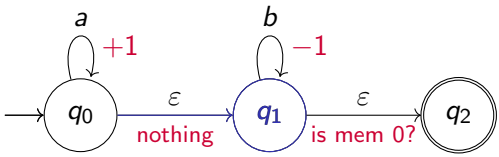


Memory: 2

Run it on input $a\mathbf{a}bb$.

PDA: intuition

Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.

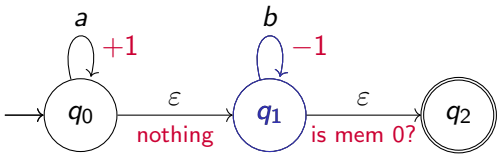


Memory: 2

Run it on input *aabb*.

PDA: intuition

Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.

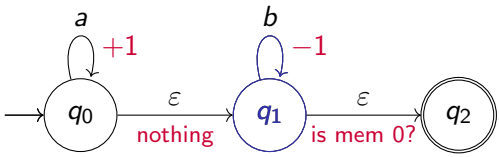


Memory: 1

Run it on input $aa\mathbf{b}b$.

PDA: intuition

Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.

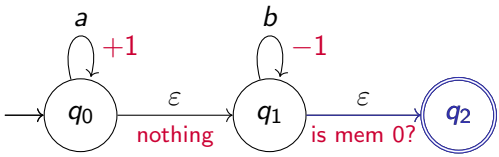


Memory: 0

Run it on input *aab*b.

PDA: intuition

Let's try to build such an *automaton with memory* for $L = \{a^n b^n \mid n \geq 0\}$.



Memory: 0

Run it on input *aabb*.

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols.

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack?

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations:

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and pop

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and pop *(just remove the first symbol of the word)*.

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and pop *(just remove the first symbol of the word)*. We will use a special alphabet for the stack and then represent the stack content as a word over that alphabet.

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and pop *(just remove the first symbol of the word)*. We will use a special alphabet for the stack and then represent the stack content as a word over that alphabet.

Each transition of the PDA will then now:

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and pop *(just remove the first symbol of the word)*. We will use a special alphabet for the stack and then represent the stack content as a word over that alphabet.

Each transition of the PDA will then now:

- read an input symbol (or ε) as in NFA,

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and pop *(just remove the first symbol of the word)*. We will use a special alphabet for the stack and then represent the stack content as a word over that alphabet.

Each transition of the PDA will then now:

- read an input symbol (or ε) as in NFA,
- pop a symbol from the stack,

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and pop *(just remove the first symbol of the word)*. We will use a special alphabet for the stack and then represent the stack content as a word over that alphabet.

Each transition of the PDA will then now:

- read an input symbol (or ε) as in NFA,
- pop a symbol from the stack, (you must **always** pop something)

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and pop *(just remove the first symbol of the word)*. We will use a special alphabet for the stack and then represent the stack content as a word over that alphabet.

Each transition of the PDA will then now:

- read an input symbol (or ε) as in NFA,
- pop a symbol from the stack, (you must **always** pop something)
- push a sequence of symbols (possibly empty) back onto the stack,

PDA: toward a formal definition

Instead of a memory counter, we will use a stack to store symbols. (If you have only one symbol, then pushing is like incrementing a counter, and popping is like decrementing it.)

How to represent a stack? It's a finite sequence of symbols *(hmm... it's like a word!)* with two operations: push *(just add symbols at the beginning to the word)* and pop *(just remove the first symbol of the word)*. We will use a special alphabet for the stack and then represent the stack content as a word over that alphabet.

Each transition of the PDA will then now:

- read an input symbol (or ε) as in NFA,
- pop a symbol from the stack, (you must **always** pop something)
- push a sequence of symbols (possibly empty) back onto the stack,
- move to a new state as in NFA.

PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,

PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,

PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,

PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,

PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,
- an initial stack symbol $z \in \Gamma$,

PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,
- an initial stack symbol $z \in \Gamma$,
- a transition relation $\Delta : \Sigma \times Q \to \mathcal{P}(Q \times \Gamma^*)$.

PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

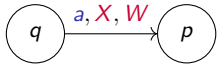
- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,
- an initial stack symbol $z \in \Gamma$,
- a transition relation $\Delta : (\Sigma \cup \{\varepsilon\}) \times Q \times \Gamma \to \mathcal{P}_f(Q \times \Gamma^*)$.

PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,
- an initial stack symbol $z \in \Gamma$,
- a transition relation $\Delta : (\Sigma \cup \{\varepsilon\}) \times Q \times \Gamma \to \mathcal{P}_f(Q \times \Gamma^*)$.

Graphically, $(p, W) \in \Delta(a, q, X)$ is represented as:

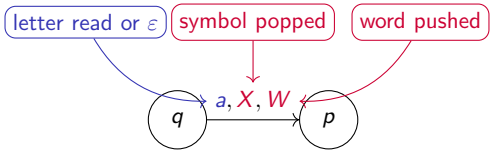


PDA: formal definition

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,
- an initial stack symbol $z \in \Gamma$,
- a transition relation $\Delta : (\Sigma \cup \{\varepsilon\}) \times Q \times \Gamma \to \mathcal{P}_f(Q \times \Gamma^*)$.

Graphically, $(p, W) \in \Delta(a, q, X)$ is represented as:



Evolution of the stack during a computation

The symbol z will only ever be used to mark the bottom of the stack.

Evolution of the stack during a computation

The symbol z will only ever be used to mark the bottom of the stack. So the word z represents the "empty" stack.

Evolution of the stack during a computation

The symbol z will only ever be used to mark the bottom of the stack. So the word z represents the "empty" stack.

Initially, the stack is "empty".

Evolution of the stack during a computation

The symbol z will only ever be used to mark the bottom of the stack. So the word z represents the "empty" stack.

Initially, the stack is "empty". So it's z.

Evolution of the stack during a computation

The symbol z will only ever be used to mark the bottom of the stack. So the word z represents the "empty" stack.

Initially, the stack is "empty". So it's z.

When the PDA uses a transition that pops a symbol X from the stack and pushes back a word W,

Evolution of the stack during a computation

The symbol z will only ever be used to mark the bottom of the stack. So the word z represents the "empty" stack.

Initially, the stack is "empty". So it's z.

When the PDA uses a transition that pops a symbol X from the stack and pushes back a word W, the stack must be of the form XS (where $S \in \Gamma^*$ is the rest of the stack),

Evolution of the stack during a computation

The symbol z will only ever be used to mark the bottom of the stack. So the word z represents the "empty" stack.

Initially, the stack is "empty". So it's z.

When the PDA uses a transition that pops a symbol X from the stack and pushes back a word W, the stack must be of the form XS (where $S \in \Gamma^*$ is the rest of the stack), and after the transition, the stack becomes WS.

Example

Let's formalize the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

Example

Let's formalize the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

We have:

Example

Let's formalize the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

We have:

- $Q = \{q_0, q_1, q_2\}$,

Example

Let's formalize the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

We have:

- $Q = \{q_0, q_1, q_2\}$,
- q_0 is the initial state,

Example

Let's formalize the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

We have:

- $Q = \{q_0, q_1, q_2\}$,
- q_0 is the initial state,
- $F = \{q_2\}$,

Example

Let's formalize the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

We have:

- $Q = \{q_0, q_1, q_2\}$,
- q_0 is the initial state,
- $F = \{q_2\}$,
- $\Gamma = \{\$, |\}$,

Example

Let's formalize the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

We have:

- $Q = \{q_0, q_1, q_2\}$,
- q_0 is the initial state,
- $F = \{q_2\}$,
- $\Gamma = \{\$, |\}$,
- $z = \$$,

Example

Let's formalize the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

We have:

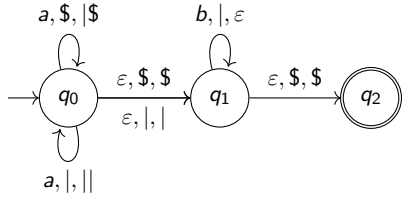
- $Q = \{q_0, q_1, q_2\}$,
- q_0 is the initial state,
- $F = \{q_2\}$,
- $\Gamma = \{\$, |\}$,
- $z = \$$,
- and Δ is given graphically:

Example

Let's formalize the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

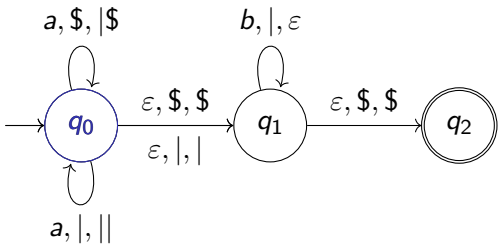
We have:

- $Q = \{q_0, q_1, q_2\}$,
- q_0 is the initial state,
- $F = \{q_2\}$,
- $\Gamma = \{\$, |\}$,
- $z = \$$,
- and Δ is given graphically:



Example (cont'd)

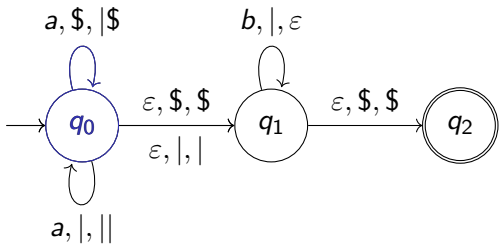
Let's run this PDA on input *aabb*.



stack: $

Example (cont'd)

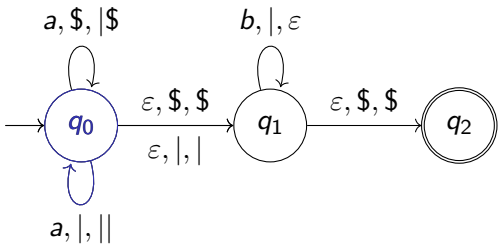
Let's run this PDA on input $aabb$.



stack: |$

Example (cont'd)

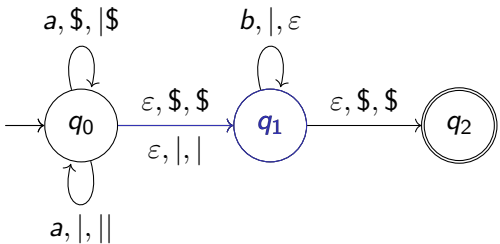
Let's run this PDA on input $aabb$.



stack: ||$

Example (cont'd)

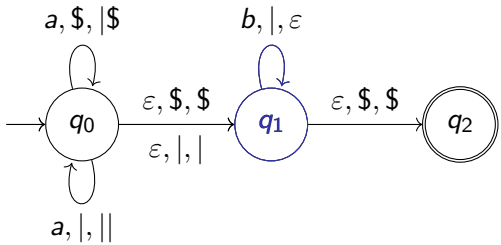
Let's run this PDA on input *aabb*.



stack: ||$

Example (cont'd)

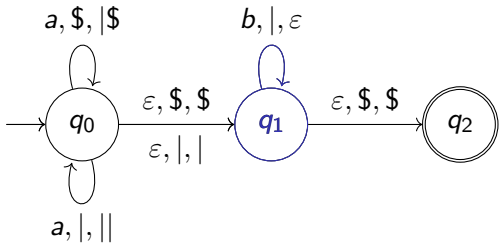
Let's run this PDA on input $aa\mathbf{b}b$.



stack: |$

Example (cont'd)

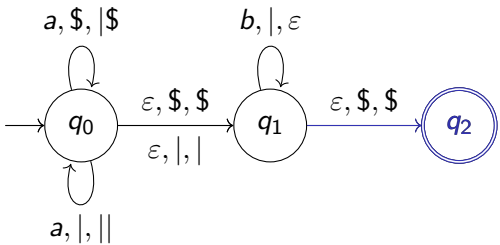
Let's run this PDA on input $aab\mathsf{b}$.



stack: $

Example (cont'd)

Let's run this PDA on input *aabb*.



stack: $

Configurations

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

Configurations

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

Call a configuration (or instantaneous description) of a PDA a triple (q, w, S) where:

Configurations

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

Call a configuration (or instantaneous description) of a PDA a triple (q, w, S) where:

- q is a state,

Configurations

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

Call a configuration (or instantaneous description) of a PDA a triple (q, w, S) where:

- q is a state,
- w is a word,

Configurations

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

Call a configuration (or instantaneous description) of a PDA a triple (q, w, S) where:

- q is a state,
- w is a word,
- S is a stack.

Configurations

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

Call a configuration (or instantaneous description) of a PDA a triple (q, w, S) where:

- q is a state,
- w is a word,
- S is a stack.

Such a configuration describes a situation where the PDA is in state q, with w as the remaining input to read, and S as the current stack content.

Computations

A computation step is a move between two configurations according to a transition of the PDA: if there is a transition $(p, W) \in \Delta(a, q, X)$, write $(q, aw, XS) \vdash (p, w, WS)$ for the corresponding computation step.

Computations

A computation step is a move between two configurations according to a transition of the PDA: if there is a transition $(p, W) \in \Delta(a, q, X)$, write $(q, aw, XS) \vdash (p, w, WS)$ for the corresponding computation step.

Write $\vdash^*$ for many (including 0) computation steps in a row.

Computations

A computation step is a move between two configurations according to a transition of the PDA: if there is a transition $(p, W) \in \Delta(a, q, X)$, write $(q, aw, XS) \vdash (p, w, WS)$ for the corresponding computation step.

Write $\vdash^*$ for many (including 0) computation steps in a row.

A computation of $\mathcal{A}$ on a word w is a sequence of computation steps from the initial configuration (q_0, w, z) to any configuration (q, ε, S):

$$(q_0, w_1 w_2 \dots w_n, z) \vdash (q_1, w_2 \dots w_n, S_1) \vdash \dots \vdash (q, \varepsilon, S)$$

Computations

A computation step is a move between two configurations according to a transition of the PDA: if there is a transition $(p, W) \in \Delta(a, q, X)$, write $(q, aw, XS) \vdash (p, w, WS)$ for the corresponding computation step.

Write $\vdash^*$ for many (including 0) computation steps in a row.

A computation of $\mathcal{A}$ on a word w is a sequence of computation steps from the initial configuration (q_0, w, z) to any configuration (q, ε, S):

$$(q_0, w_1 w_2 \dots w_n, z) \vdash (q_1, w_2 \dots w_n, S_1) \vdash \dots \vdash (q, \varepsilon, S) \quad (\text{i.e.,} (q_0, w, z) \vdash^* (q, \varepsilon, S))$$

Computations

A computation step is a move between two configurations according to a transition of the PDA: if there is a transition $(p, W) \in \Delta(a, q, X)$, write $(q, aw, XS) \vdash (p, w, WS)$ for the corresponding computation step.

Write $\vdash^*$ for many (including 0) computation steps in a row.

A computation of $\mathcal{A}$ on a word w is a sequence of computation steps from the initial configuration (q_0, w, z) to any configuration (q, ε, S):

$$(q_0, w_1 w_2 \dots w_n, z) \vdash (q_1, w_2 \dots w_n, S_1) \vdash \dots \vdash (q, \varepsilon, S) \quad (\text{i.e.,} (q_0, w, z) \vdash^* (q, \varepsilon, S))$$

The computation is successful if $q \in F$.

Language recognized by a PDA

$\mathcal{A}$ accepts a word w if there is a successful computation of $\mathcal{A}$ on w.

Language recognized by a PDA

$\mathcal{A}$ accepts a word w if there is a successful computation of $\mathcal{A}$ on w.

The language recognized by $\mathcal{A}$ is the set of all the words it accepts:

$$\mathsf{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \exists S, \exists q \in F, (q_0, w, z) \vdash^* (q, \varepsilon, S)\}$$

Exercise

Make a PDA that recognizes the language of binary words with the same number of 0s and 1s.